

PRÁCTICA

1) (3 Puntos) A partir del esquema dado, escriba las sentencias SQL necesarias para resolver los ítems propuestos:

Carreras(cod_carrera: integer, nombre: string, duracion: integer, alumnos: integer)

Alumnos(numa: integer, nombrea: string, carrera: integer, nivel: string, edad: integer)

CF: carrera → Carreras(cod_carrera)

Cursos(nombre: string, impartido en: string, aula: string, idp: integer, max_alumnos: integer)

CF: idp → Profesor(idp)

Matriculado(numa: integer, nombrec: string)

CF: numa → Alumnos(numa); nombrec → Cursos(nombre)

Profesores(idp: integer, nombrep: string, iddep: integer, sueldo: real)

a) Crear los triggers necesarios para:

- i. al intentar matricular un alumno a un curso, controle que la cantidad de alumnos matriculados en ese curso no sobrepase el máximo de alumnos permitidos (valor “*max_alumnos*” en la tabla “**Alumnos**”), cancelando la operación y emitiendo un error en caso contrario.

`delimiter //`

```
CREATE TRIGGER control_max
BEFORE INSERT ON Matriculado /* PODRÍA CONTEMPLARSE TAMBIÉN AGREGAR OTRO CON
"BEFORE UPDATE" */
FOR EACH ROW
BEGIN
    SET @cant_alumnos = 0;
    SET @max = 0;
    SELECT COUNT(*) INTO @cant_alumnos
        FROM Matriculado m
        WHERE m.nombrec = NEW.nombrec;
    SELECT c.max_alumnos INTO @max
        FROM Cursos c
        WHERE c.nombre = NEW.nombrec;
    IF @cant_alumnos >= @max THEN
    BEGIN
        SET @mensaje = CONCAT('Se superó el máximo de alumnos permitidos en
', NEW.nombrec);
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = @mensaje;
    END;
    END IF;
END//

DELIMITER ;
```

- ii. Asumiendo que el campo “*alumnos*” de la tabla “**Carreras**” contabiliza la cantidad de alumnos inscriptos en cada carrera, actualizar el contador cada vez que se da de alta un alumno en una carrera determinada;

`delimiter //`

```
CREATE TRIGGER actualizar_cant
AFTER INSERT ON Alumnos /* PODRÍA CONTEMPLARSE TAMBIÉN AGREGAR OTRO
```

```
CON "AFTER UPDATE" */
FOR EACH ROW
BEGIN
    UPDATE Carreras
        SET alumnos = alumnos + 1
        WHERE cod_carrera = NEW.carrera;
END//
```

```
delimiter ;
```

- b) Crear una rutina almacenada que, dado código de una carrera, liste los detalles de los alumnos inscriptos en la misma y en qué cursos están matriculados, ordenados por nombre de alumno y nombre de curso.

```
delimiter //
```

```
CREATE PROCEDURE ListarAlumnos(IN carrera SMALLINT)
BEGIN
    SELECT a.nombrea, a.nivel, a.edad, m.nombrec
    FROM Alumnos a, Matriculado m
    WHERE a.numa = m.numa AND a.carrera = carrera
    ORDER BY a.nombrea, m.nombrec;
END//
```

```
delimiter ;
```

2) (3 puntos) Seguridad.

Usando el esquema de la BD “FACULTAD” dado en el ejercicio anterior, escriba las sentencias SQL que permitan realizar las siguientes acciones:

- a) Crear un nuevo usuario llamado Pedro y asignar la contraseña “Pedro123”. La contraseña vence cada 90 días.
- ```
CREATE USER 'Pedro'@'%' IDENTIFIED BY 'Pedro123' PASSWORD EXPIRE INTERVAL 90 DAY;
```

- b) Asignar permiso para realizar únicamente consultas de visualización de las tablas “Profesores” y “Cursos”, excepto sobre el campo “sueldo” de la tabla “Profesores” en la base de datos “FACULTAD”.

```
GRANT SELECT ON FACULTAD.Cursos TO 'Pedro'@'%';
GRANT SELECT(idp, nombrep, iddep) ON FACULTAD.Profesores TO 'Pedro'@'%';
```

- c) Crear un usuario que tenga los privilegios necesarios para respaldar y restaurar la base de datos “FACULTAD”.

```
CREATE USER 'respaldo'@'%' IDENTIFIED BY 'Respaldo123';
GRANT SELECT, CREATE, ALTER, DROP, INSERT ON FACULTAD.* TO 'respaldo'@'%';
/* ESTOS PRIVILEGIOS AL MENOS PARA RESGUARDAR DATOS Y ESTRUCTURAS DEL EJERCICIO Y PODER VOLVER A CREAR Y CARGAR */
```

## 3) (4 puntos) Respaldo y restauración.

- a) Plantee un esquema de respaldos para la base de datos “FACULTAD” del ejercicio 1.

DEBERÍA HABER UN PLANTEO SOBRE POSIBLES VOLÚMENES DE DATOS/TRANSACCIONES, Y QUE LA CANTIDAD Y TIPO DE RESPALDOS DIARIOS/SEMANALES SEAN COHERENTES CON DICHOS VOLÚMENES.

- b) Utilizando el usuario creado en 2.c, escriba las sentencias/comandos, y aclaraciones que considere necesarias, tanto para hacer respaldos (completos y/o incrementales) como para recuperar los datos en caso de una contingencia.

PARA REALIZAR UN RESPALDO LÓGICO COMPLETO QUE REINICIE LOS BINARY LOGS PARA LLEVAR RESPALDOS INCREMENTALES:

```
mysql-dump -u respaldo -p --single-transaction --flush-logs
--source-data=2 FACULTAD > facultad.sql
```

PARA RECUPERAR EL RESPALDO COMPLETO:

```
mysql -u respaldo -p < facultad.sql
```

PARA RECUPERAR TRANSACCIONES DE LOS BINARY LOGS:

```
mysqlbinlog binlog.000009 binlog.000010 | mysql -u respaldo -p
```

SE RECUPERA LOS BINLOGS NECESARIOS HASTA EL PUNTO DE FALLO, Y DE SER NECESARIO SE AGREGAN/DESCARTAN FRAGMENTOS ESPECÍFICOS MEDIANTE LAS OPCIONES --start-position / --stop-position

## TEORÍA (2 puntos)

- 4) Programabilidad en bases de datos. Procedimientos Almacenados y Disparadores. Definición. Similitudes y Diferencias.
- 5) Escribe conceptos de Transacciones y concurrencia. ¿cómo soluciona un motor relacional los problemas de concurrencia? ¿Qué es el Schedule o Planificación?.
- 6) Copias de seguridad. Concepto. Responsabilidades de DBA. Factores que influyen en la duración de la recuperación. Lineamientos generales. Tipos de copias de seguridad. Recuperación. Determinación de opciones de recuperación. Tipos de recuperación.
- 7) Control de Concurrencia. Defina las violaciones causadas por cada una de estas circunstancias: lectura sucia, lectura de datos no repetibles y sobreescritura de datos no consolidados.